APPENDIX A

A Topic Template

Every topic in this roadmap is one Markdown file with the same shape, so you always know where to find the explanation, the diagram, the hands-on guide and the tasks.

File naming: `NN-topic-slug.md` inside `level-NN-level-slug/`. The last file in each level is `99-capstone-project.md`, which ties the topics of that level together.

Required sections (in this order)

MARKDOWN

```
# L{level}.{topic} — {Topic Title}

> **Level {level}: {Level Title}** · Topic {topic} of {total} · ⏱ ~{N} hours ·
  Difficulty: {Beginner|Intermediate|Advanced}

## 1. What it is
Plain-English explanation in 1-2 short paragraphs. Define every acronym on first use.

## 2. Why it matters in DevOps
What breaks, costs money or wakes someone up at 3 a.m. when this is misunderstood.

## 3. Real-world system example
A named, real, publicly documented system or company (e.g. Netflix, GitHub, Stripe,
  Cloudflare, Shopify,
Airbnb, Spotify, Google, AWS itself, Kubernetes, Linux kernel) and how this topic shows up
  there.
Only state facts you are confident are publicly documented. Do not invent numbers, dates
  or internal details.
If a precise claim is uncertain, describe the pattern generically ("large streaming
  platforms typically…").

## 4. Diagram
At least one Mermaid diagram showing the real mechanism (flow, sequence, architecture or
  state machine).
Add a second diagram when it helps (e.g. "before vs after", "failure path").

## 5. How it works
Key concepts, a comparison table or a short reference of the important commands/options.

## 6. Step-by-step guide
```

```
Numbered steps. Every command is copy-pasteable, in a fenced block with a language tag,
  with the
expected output or what to look for. State prerequisites first (OS, tools, accounts,
  cost).
Use a safe sandbox (a local VM/container, Docker, kind, LocalStack, free tier) wherever
  possible.

## 7. Real-world scenario
A short story (an outage, a migration, a scaling event) showing the topic applied under
  pressure:
symptom → investigation → fix → lesson.

## 8. Common mistakes and best practices
Two short lists: "Don't" and "Do".

## 9. Tasks
A checklist of hands-on tasks, easiest first. Each task has an ID, a goal, steps and a
  "Done when" check.

## 10. Self-check questions
4-6 questions that a senior engineer would expect you to answer without notes.
Put answers inside a `<details>` block.

## 11. Further reading
Official documentation and well-known books only. Never guess a deep URL; link to a
  documentation root
or name the document/book instead.
```

Task format

MARKDOWN

```
- [ ] **T1 — Short imperative title** *(≈20 min, beginner)*
  - **Goal:** what you will have achieved.
  - **Steps:**
    1. …
    2. …
  - **Done when:** an observable, verifiable result (a command's output, a URL that
  responds, a metric that appears).
```

Rules for tasks:

- 4-6 tasks per topic, progressing from "follow along" → "modify" → "break it and fix it" → "explain / automate it".
- The last task is always a **Prove it** task: do it without the guide, then explain it out loud or in writing.

- Every task is independently verifiable. "Understand X" is not a task; "Show X with this command and explain the output" is.
- Anything that may cost money says so up front and ends with a cleanup step.

Diagram rules (Mermaid)

- Use fenced ```` ```mermaid ```` blocks; they render natively on GitHub.
- Allowed types: `flowchart`, `sequenceDiagram`, `stateDiagram-v2`, `classDiagram`, `erDiagram`, `gantt`, `timeline`. Prefer `flowchart` and `sequenceDiagram`.
- Wrap any label that contains spaces plus punctuation such as `( ) / : , . | #` or quotes in double quotes: `A["Load balancer (ALB)"]`. Never put raw parentheses or slashes in an unquoted label.
- Use `<br/>` for line breaks inside labels, not `\n`.
- Do not use the words `end`, `graph` or `subgraph` as node ids. Node ids are short ASCII (`A`, `lb`, `db1`).
- Keep diagrams readable: ≤ 15 nodes each.
- Edge labels: `A ⟶|label| B` (no quotes needed for simple words; quote if punctuation).